Chris Sun

fzgt320@gmail.com | (+61) 0400 297 972 | github.com/Fzgt

TECHNICAL SKILLS

Programming Languages: TypeScript, JavaScript, Python, Go

Frontend: React, Next.js, Vue, Angular, Tailwind, TanStack Query, Zustand, R3F, Three.js

Backend & Data: Node.js, NestJS, FastAPI, Django, PostgreSQL, pgvector, MongoDB, WebSockets

Engineering: Playwright, Jest, Vitest, Webpack, Vite, Jenkins, GitHub Actions, Docker, AWS, Cloudflare

AI: LLM Integration, RAG, Structured Outputs, Embeddings, Vector Search, Streaming, Evals

EDUCATION

University of Technology Sydney (UTS) Aug 2024 – Jun 2026
Master of Information Technology, Enterprise Software Development *Sydney, Australia*

Shandong Technology and Business University Sep 2017 – Jun 2021
Bachelor of Science, Information and Computing Science *Shandong, China*

EXPERIENCE

Beyz Jan 2024 – Oct 2024
Full-stack Engineer *Canada, Remote*

– Built the real-time interview copilot end to end: tab and system audio captured through getDisplayMedia, streamed over a WebSocket to Deepgram's live recognizer, with interim and finalized transcripts reconciled in the client and utterance-end events triggering GPT-4o answers streamed back token by token.

– Kept the transcription credential out of the browser by minting 60-second, usage-scoped keys per session behind a plan gate, held the socket alive across microphone pauses, and logged per-request token usage that backs per-plan quotas; a four-state machine drops overlapping generations.

– Owned the paid conversion path — Stripe Checkout, trials and lifetime purchases on Next.js, tRPC and Drizzle — with plan eligibility enforced in typed server actions and entitlements written only by signature-verified webhooks across six subscription events, none of which can downgrade a lifetime purchase.

– Sole author of a NestJS and Fastify service for subscription emails and scheduled payment reminders — Dockerized, cron-triggered, deployed to EC2 under PM2 through GitHub Actions — and of the repo's first CI gate: lint, typecheck and build on every push.

BioMap Apr 2022 – Oct 2022
Frontend Engineer *Beijing, China*

– Developed the LIMS platform in Vue 3 and TypeScript for sample management and SOP workflows, integrating the Luckysheet SDK and a CLI for one-command project scaffolding.

– Led the React 18 and TypeScript architecture for Digital OS, the team's core consumer-facing product, enforcing feature boundaries with custom ESLint rules.

– Layered TanStack Query in three tiers — API functions, queryOptions factories, typed hooks — single-source cache keys, mutation-driven invalidation and hover prefetch: detail pages open instantly, with no request at all.

– Built a design system on Radix UI, Storybook and CVA, plus an MSW mock layer that let frontend delivery run independent of backend progress.

Baidu Jul 2021 – Apr 2022
Frontend Engineer *Beijing, China*

– Delivered the Xiaodu App's H5 smart-home module in Vue 2, adding native iOS/Android sharing via jsBridge, device detail pages and analytics tracking.
– Built a virtualized React feed serving 5M+ daily page views on a custom useVirtualList hook over IntersectionObserver: live DOM nodes stay near 100 at any scroll depth, scroll from 30 to a stable 60 FPS.
– Built the withdrawal client on a path peaking above 2,000 QPS: request locking drops in-flight duplicates, client-generated idempotency keys survive reloads, and uncertain outcomes surface as failures, never successes.
– Led a Webpack 3-to-5 migration across 10+ incompatible packages, extracting vendor and runtime chunks and turning on persistent caching — bundle size down ~15%, local build times down 70% on warm builds.
– First to diagnose a P2 white screen on Haokan Video's search page (30M+ DAU): mismatched Node versions regenerated package-lock.json, pulling a breaking minor bump — version locks and release checks followed.

PROJECTS

Periplus — Evidence-Grounded LLM Travel Research *Python · FastAPI · React · pgvector*

– Built a six-stage agentic workflow — research, verification, planning, writing, editing and illustration — where the orchestrator knows only typed stage contracts, carried through to the React client as types generated from the OpenAPI schema.
– Split evidence from judgment: the research stage emits claims bound to exact source quotes, a separate verification stage rules on them, and application code — not prompts — enforces citation membership and freshness windows. A claim with no supporting quote never advances.
– Persisted run state, stage artifacts and a pgvector evidence cache in PostgreSQL under per-run token and time budgets. A run that dies mid-pipeline resumes from the last completed stage.
– Put a scripted seam behind every external dependency — model provider, search, page fetch and clock — so the whole system runs with no API keys and no network: 406 server tests, about a second end to end.
– Scored the pipeline against a golden set, not by inspection. Each case freezes the brief, the retrieved pages and the clock, then asserts ranges rather than literal output on evidence-gap rate, citation integrity and token cost. A prompt or control-flow change lands as a before/after diff, every metric marked better or worse.

WinterFell — WebGPU Neon Runner *React Three Fiber · Three.js · WebGPU · Cannon · TypeScript*

– Built the bloom pass on the three.js TSL node pipeline, which a classic WebGLRenderer cannot execute. A single WebGPURenderer drives WebGPU where present and its own WebGL2 backend where absent; the look survives the fallback instead of quietly degrading.
– Collapsed each obstacle section into two InstancedMeshes sharing one matrix set: a headless Chromium harness sampling renderer stats every frame put draw calls down from 129 to 76 and triangles submitted up 2.6×, one bounding volume per mesh being the price of losing per-instance frustum culling.
– Swept the live generator across 1,020 generated layouts, testing every step against the craft's lateral reach at that level's speed — every layout had a continuous path through it, no gap narrower than 15 units, against a random-scatter control.
– Traced a freeze landing exactly on each palette change to distance fog: declared as JSX it rebuilt the fog object, and fog belongs to a material's program key, so the renderer recompiled every material in the scene inside one frame — a probe counting pipelines built per frame confirmed it; one mutated fog object removed it.